When columns have values that uniquely identify individuals they're considered to be sensitive. A good security practice is to limit access to sensitive data. To do this, you'll build a *scrubbing* process that replaces sensitive values with data that is representative of real data, but benign.

A design goal of the scrubbing techniques you'll use is that they'll work quickly even on millions of rows. This provides an opportunity to discuss some of the ways that row modifications can slow down in PostgreSQL.

Another design goal is that the *scrubbed* data maintains the statistical properties of the original data as much as possible. This way your query performance analysis in your Performance database will very closely match your production database.

Performance Database Terminology



- Schema — In PostgreSQL it's a namespace
- Scrubbing — Replacing sensitive text in your database fields, as a good security practice
- Functions — Database functions in SQL or the built-in procedural language pl/pgSQL
- Procedures — An extended form of functions
- Cardinality — Unique values in a column

The scrubbing process works with all modern versions of PostgreSQL and has no external dependencies. To achieve that level of flexibility, this means you'll be building all of the pieces yourself!

To get started, first you'll download a copy of a pre-made data dump. Read on to find out how to do that.

Downloading Production Data Dumps

The Riders and Drivers in Rideshare were generated using fake values from the Faker gem[1] which produces values designed to look real.

A pre-made dump of one million rows was made using `pg_dump` as below. You don't need to run this now, it's just here to show you what was done earlier.

sh/dump_rideshare_database.sh

```
# create the dump file
pg_dump --dbname rideshare_development \
```

1. https://github.com/faker-ruby/faker